

ALGEBRAIC FOUNDATIONS OF LOGIC PROGRAMMING, I: THE DISTRIBUTIVE LATTICE OF LOGIC PROGRAMS¹

Anil HIRANI and V.S. SUBRAHMANIAN

MS 1-40, SUN Microsystems, 2550 Garcia Avenue, Mountain View, CA 94043, USA
and

Department of Computer Science, University of Maryland, College Park, MD 20742, USA

V.S. Subrahmanian

Department of Computer Science
University of Maryland
College Park, MD 20742, U.S.A.

Given a logic program P , the operator T_P associated with P is closely related to the intended meaning of P . Given a first order language L that is generated by finitely many non-logical symbols, our aim is to study the algebraic properties of the set $\{T_P \mid P \text{ is a general logic program in language } L\}$ with certain operators on it. For the operators defined in this paper the resulting algebraic structure is a bounded distributive lattice. Our study extends (to the case of general logic programs), the work of Mancarella and Pedreschi who initiated a study of the algebraic properties of the space of pure logic programs. We study the algebraic properties of this set and identify the ideals and zero divisors. In addition, we prove that our algebra satisfies various *non-extensibility* conditions.

1 Introduction

Given a first order language, and a collection of logic programs in that language, in what ways can we *conjoin* these programs together? Any systematic method of conjoining two programs may be viewed as an operator that maps a pair of programs to a program. Thus, if we have a collection C of programs, and a finite set $\{f_1, \dots, f_n\}$ of *admissible* operators for conjoining programs, (each of these operators may correspond to different operations on programs), we notice that we have an algebra $\langle C, f_1, \dots, f_n \rangle$. For instance, these operators may correspond to program transformations and/or to simple ways of conjoining programs like taking the union of two programs. As we vary our choice of C and our choice of what the admissible operators are, the properties of our algebra change. Our aim in this paper is to study the algebraic properties of the set of logic programs expressible in a first order language generated by finitely many non-logical symbols along with some operators on this set.

Mancarella and Pedreschi [17] took the first steps towards the problem of defining an algebraic semantics for logic programming. Associated with a program P is an operator T_P that maps

¹Address Correspondence to V.S. Subrahmanian.

Herbrand interpretations to Herbrand interpretations. It has been argued by various individuals [14] that T_P characterizes the meaning of P in an elegant way. Let $\Phi(L) = \{T_P \mid P \text{ is a pure logic program in the language } L\}$. What Mancarella and Pedreschi did was to define two operators $\oplus_{MP}$ and $\otimes_{MP}$ of type $\Phi(L) \times \Phi(L) \rightarrow \Phi(L)$. They then proved various properties of the algebra induced on $\Phi(L)$ by these two operations.

Our aim, in this paper, is to develop a comprehensive theory of the algebraic properties of the set $\Psi(L) = \{T_P \mid P \text{ is a general logic program in } L\}$ with certain binary operations on this set. Two binary operations, $\oplus$ and $\otimes$ are defined on $\Psi(L)$. These operators are essentially the same as those of Mancarella and Pedreschi except that they now apply to general logic programs. We then study the algebraic properties of $\Psi(L)$. We show that our algebra and Mancarella and Pedreschi's algebra [17] are bounded distributive lattices.

This work improves upon the work of Mancarella and Pedreschi [17] in many ways:

1. it is applicable to general logic programs, whereas their approach is applicable to pure logic programs only.
2. the Mancarella-Pedreschi algebra is a subalgebra of our algebra. Hence, our approach may be viewed as an extension of theirs.
3. we show that there is no complementation operator $\neg$ such that $\Phi(L)$ together with the operators $\oplus_{MP}, \otimes_{MP}$ and $\neg$ forms a Boolean algebra. This significantly improves a result of Mancarella and Pedreschi [17] which states that a *specific* definition of $\neg$ does not yield a Boolean algebra.
4. we show that $\Psi(L)$ forms a bounded distributive lattice (under the operations $\oplus$ and $\otimes$). However, in general, there may be zero divisors (defined later). We derive necessary and sufficient conditions for a T_P operator to be a zero divisor.
5. we show what these results mean from the point of view of logic program compositionality and subsumption equivalence (cf. Maher [14]) of general logic programs. This extends Maher's earlier results on subsumption equivalence of *pure* logic programs.

This is the first of an n -part ($n \geq 3$) series of papers on the algebraic aspects of logic programming semantics. Our primary aim here is to understand various properties of the space of logic programs. There is a crucial need for this. For instance, the study of logic program *compositionality* is essentially an algebraic one (cf. Mancarella and Pedreschi [17]). A logic program transformation is a map from the space of programs in language L_1 to the space of programs in (a possibly different) language L_2 . Thus, algebra forms a natural framework for the study of such phenomena. In Part II of this series, we will study the algebraic properties of the space of multi-valued logic programs. This would be a natural step in the research directions forged in [9,10,6,7,20]. In [3], it was shown that the space of all reachable models of a logic program form a compact Hausdorff space under a natural topology called the *query topology*. The Stone Representation Theorem allows us to make the connection that the clopen sets in the query topology form a Boolean Algebra. This forms the subject of Part III of this series.

2 Distributive Lattices of Logic Programs

We assume that the reader is familiar with standard notions in abstract algebra. The reader is referred to Grätzer [11] for the formal terminology used here. We assume that there is a first order language L satisfying the following conditions:

(L1) L has finitely many constant, function and predicate symbols.

(L2) the Herbrand Base B_L of L is non-empty.

Following Mancarella and Pedreschi [17], we define two binary operators $\oplus$ and $\otimes$ on $\Psi(L)$ as follows: For all Herbrand interpretations $I \subseteq B_L$:

$$(T_P \oplus T_Q)(I) = T_P(I) \cup T_Q(I).$$

$$(T_P \otimes T_Q)(I) = T_P(I) \cap T_Q(I).$$

Proposition 1 1. $\langle \Psi(L), \oplus, \otimes \rangle$ and $\langle \Phi(L), \oplus, \otimes \rangle$ are both bounded distributive lattices.

2. $\langle \Phi(L), \oplus, \otimes \rangle$ is a $\{0, 1\}$ -sublattice of $\langle \Psi(L), \oplus, \otimes \rangle$. □

Throughout this paper, we denote the least element and the greatest element of any bounded distributive lattice by the (generic) symbols $\mathbf{0}$ and $\mathbf{1}$ respectively. The context will distinguish the underlying bounded distributive lattice. For the bounded distributive lattices $\langle \Phi(L), \oplus, \otimes \rangle$ and $\langle \Psi(L), \oplus, \otimes \rangle$, $\mathbf{0}$ is $\lambda I. \emptyset$ while $\mathbf{1}$ is $\lambda I. B_L$. It is easy to see that L contains at least one pure logic program P and at least one pure logic program Q such that $T_P = \mathbf{0}$ and $T_Q = \mathbf{1}$ ¹.

Definition 1 A bounded distributive lattice $\langle D, +, \cdot \rangle$ is said to have a Boolean extension iff there is a unary operator $\neg : D \rightarrow D$ such that $\langle D, +, \cdot, \neg, \mathbf{0}, \mathbf{1} \rangle$ is a Boolean algebra (Here $\mathbf{0}$ is the least element of D and $\mathbf{1}$ is the greatest element of D).

We assume here that the reader is familiar with the notion of a Boolean Algebra, details of which may be found in Sikorski [19] or Henkin et al [12, p.161] and the definition of a ring which may be found in any elementary abstract algebra text (cf. Atiyah and Macdonald[1]).

Suppose L, L' are first order languages such that the non-logical symbols of L are included in those of L' . If X is a set of formulas in language L' , then we use the notation $X|_L$ to denote the set of formulas in X that are formulas of L . The following lemma is well known.

Lemma 1 Suppose I is an r.e. subset of B_L , and P is a pure logic program. Then $T_P(I)$ is an r.e. subset of B_L . Moreover, if L' contains all the non-logical symbols of L , and P' is a pure logic program in language L' , then $T_{P'}(I)|_L$ is also an r.e. subset of B_L . □

Theorem 1 (First Boolean Non-Extensibility Theorem.) Suppose B_L is infinite. Then: the bounded distributive lattice $\langle \Phi(L), \oplus, \otimes \rangle$ does not have a Boolean extension.

¹if Axiom (L1) were not assumed to hold for the language L , then there may be no logic program Q such that $T_Q = \mathbf{1}$.

Proof. Suppose $\langle \Phi(L), \oplus, \otimes \rangle$ has a Boolean extension, and the associated unary operator is $\neg$. Then for every pure logic program P , there is a pure logic program, denoted $\neg P$ such that for all Herbrand interpretations I , the following two properties hold:

$$(I) \quad (T_P \oplus T_{\neg P})(I) = B_L \text{ and}$$

$$(II) \quad (T_P \otimes T_{\neg P})(I) = \emptyset.$$

As B_L is infinite, there is an interpretation I_0 that is a strictly r.e.² subset of B_L . Select a pure logic program P_0 such that $[lf_P(T_{P_0})]_L = I_0$. This is possible, by results of Blair[5] because each r.e. subset of B_L is the success set of some pure logic program in language L' where L' contains all the non-logical symbols of L . By (I) and (II) above, it follows that:

$$[T_{P_0}(I_0) \cup T_{\neg P_0}(I_0)]_L = B_L$$

and

$$[T_{P_0}(I_0) \cap T_{\neg P_0}(I_0)]_L = \emptyset$$

Hence,

$$[T_{\neg P_0}(I_0)]_L = B_L - T_{P_0}(I_0)$$

As I_0 was selected to be the least fixed-point of T_{P_0} , we know that $[T_{P_0}(I_0)]_L = I_0$ is strictly r.e. Thus, $[T_{\neg P_0}(I_0)]_L$ is a non-r.e. subset of B_L . This contradicts Lemma 1. $\square$

Observe here that Lemma 1 does not hold for general logic programs. When P is not a pure logic program, it is perfectly possible that I is r.e., while $T_P(I)$ is not r.e. To see this, consider the following example.

Example 1 Suppose P is a logic program containing a unary predicate symbol p such that $I \subseteq \{p(t) \mid p(t) \in B_L\}$ is a strictly r.e. (i.e. non-recursive, but r.e.) subset of B_L . Consider now the logic program Q containing a unary predicate symbol q that occurs nowhere in P such that

$$Q = P \cup \{q(X) \leftarrow \neg p(X)\}$$

Then for any ground atom $q(t)$, $q(t)$ is in $T_P(I)$ iff $p(t) \notin I$. As I is strictly r.e., the set $\{q(t) \mid q(t) \in T_P(I)\}$ is not r.e.

Also observe that the proof of the First Boolean Non-Extensibility Theorem does not hold if B_L is finite. We show below a much stronger result.

Theorem 2 (Second Boolean Non-Extensibility Theorem.) The bounded distributive lattice $\langle \Phi(L), \oplus, \otimes \rangle$ has a Boolean extension iff B_L is empty.

Proof. It is easy to see that when B_L is empty, then $\langle \Phi(L), \oplus, \otimes \rangle$ is the trivial bounded distributive lattice consisting of the single element 0 . This is because when $B_L = \emptyset$, the only program expressible in L is the empty program. Hence, we only need to show the other direction.

²Throughout this paper the expression “strictly r.e.” will be considered synonymous with “r.e. but not recursive.”

Suppose $\langle \Phi(L), \oplus, \otimes \rangle$ has a Boolean extension and B_L is non-empty. Then we derive a contradiction as follows: Let $A \in B_L$. Then there is a pure logic program P and two interpretations I_1, I_2 of L such that:

(I) $I_1 \subset I_2$ (here $\subset$ is strict inclusion) and

(II) $T_P(I_1) \subset T_P(I_2)$.

To see that there is at least one triple (P, I_1, I_2) satisfying (I) and (II) above, observe that we can take $P = \{A \leftarrow A\}$, $I_1 = \emptyset$ and $I_2 = \{A\}$. So let (P, I_1, I_2) be one such triple. By our assumption that $\langle \Phi(L), \oplus, \otimes \rangle$ has a Boolean extension, there is a pure logic program $\neg P$ in L such that for all interpretations I of L :

(III) $T_P(I) \cup T_{\neg P}(I) = B_L$ and

(IV) $T_P(I) \cap T_{\neg P}(I) = \emptyset$.

Hence:

(V) $T_{\neg P}(I) = B_L - T_P(I)$.

As both P and $\neg P$ are pure logic programs, we know that T_P and $T_{\neg P}$ are monotone. Since $I_1 \subseteq I_2$, we have that:

(VI) $T_{\neg P}(I_1) \subseteq T_{\neg P}(I_2)$

But by (V) we know that $T_{\neg P}(I_1) = B_L - T_P(I_1)$ and $T_{\neg P}(I_2) = B_L - T_P(I_2)$. Thus, as $T_P(I_1) \subset T_P(I_2)$, we know that

(VII) $(B_L - T_P(I_1)) \supset (B_L - T_P(I_2))$

(Here $\supset$ denotes strict superset). From (V) and (VII), we know that

(VIII) $T_{\neg P}(I_1) \supset T_{\neg P}(I_2)$

This contradicts the monotonicity of $T_{\neg P}$. □

Definition 2 A bounded distributive lattice $\langle D, +, \cdot \rangle$ is said to have a *ring-extension* iff there is a ring $\langle D', +', \cdot' \rangle$ such that: (1) $D \subseteq D'$ and (2) for all $x, y \in D$, $x + y = x +' y$ and $x \cdot y = x \cdot' y$ and (3) the least element of D (w.r.t. the special partial ordering on D) and the least element of D' (w.r.t. the special partial ordering on D') are the same and (4) the greatest element of D (w.r.t. the special partial ordering on D) and the greatest element of D' (w.r.t. the special partial ordering on D') are the same.

For logic programs, we have a slight extra condition: we insist that $\langle \Phi(L), \oplus, \otimes \rangle$ (resp. $\langle \Psi(L), \oplus, \otimes \rangle$) has a ring-extension $\langle \Phi(L'), \oplus, \otimes \rangle$ (resp. $\langle \Psi(L'), \oplus, \otimes \rangle$) iff in addition to conditions (1) – (4) above, $L \subseteq L'$.

Definition 3 A clause $A \Leftarrow L_1 \& \dots \& L_n$ in language L is said to be *body-inconsistent* iff

$$(\exists x_1, \dots, x_m)(\exists y_1, \dots, y_k)(L_1 \& \dots \& L_n)$$

has no Herbrand model (w.r.t. L). Here $y_1, \dots, y_k$ are all the variable symbols that occur in the body of the above clause, but not in its head. $x_1, \dots, x_m$ are all the *other* variables that occur in the above clause. A logic program P is *body inconsistent* iff each clause in P is body inconsistent. A logic program is *body consistent* iff it is not body inconsistent.

Lemma 2 Suppose $B_L \neq \emptyset$ and P is a general logic program in B_L . Then: $T_P(I) = \emptyset$ for all interpretations I iff P is body inconsistent.

Proof. ($\rightarrow$) Suppose that $T_P(I) = \emptyset$ for all interpretations I . We show that P is body-inconsistent. If $P = \emptyset$, then P is trivially body inconsistent, so assume that P contains at least one clause $C \equiv$

$$A \Leftarrow B_1 \& \dots \& B_k \& \neg D_1 \& \dots \& \neg D_m$$

which is body-consistent. We derive a contradiction as follows. First, let $y_1, \dots, y_s$ be all the variables occurring in the body of the above clause, but not in the head. Let $x_1, \dots, x_r$ be all other variable symbols occurring in this clause. As C is body consistent, there is an $I \subseteq B_L$ that is a model of:

$$(\exists x_1, \dots, x_r)(\exists y_1, \dots, y_s)(B_1 \& \dots \& B_k \& \neg D_1 \& \dots \& \neg D_m)$$

Then, for some substitution $\sigma = \{x_1/t_1, \dots, x_r/t_r\}$ of ground terms t_i of L for the variable symbols x_i , it follows that I is a model of

$$(\exists y_1, \dots, y_s)((B_1 \& \dots \& B_k \& \neg D_1 \& \dots \& \neg D_m)\sigma)$$

But then $A\sigma \in T_P(I)$, (note that $A\sigma$ is ground) thus contradicting the assumption that $T_P(I) = \emptyset$ for all $I \subseteq B_L$. This completes one direction of the proof.

($\leftarrow$) Suppose P is body-inconsistent and $A \in T_P(I)$ for some $A \in B_L$ and some $I \subseteq B_L$. Then there is some clause $C \in P$ and a substitution σ such that $C\sigma \equiv$

$$A \Leftarrow B_1 \& \dots \& B_k \& \neg D_1 \& \dots \& \neg D_m$$

is a ground instance of C and $I \models (B_1 \& \dots \& B_k \& \neg D_1 \& \dots \& \neg D_m)$. Let *Body* be the body of C – hence, we know that

$$I \models (\text{Body})\sigma.$$

Let $\sigma_b = \{y/t \in \sigma \mid y \text{ occurs in the body of } C \text{ but not in the head of } C\}$ and let σ_o denote $(\sigma - \sigma_b)$. Then $I \models (\exists x_1, \dots, x_r)(\text{Body})\sigma_b$ where $x_1, \dots, x_r$ are all the variables in the domain of σ_o . This contradicts the assumption that I is body-inconsistent. $\square$

Example 2 Note that the empty logic program is trivially body-inconsistent. Similarly, the program:

$$q \Leftarrow p \& \neg p$$

is body inconsistent. There are four interpretations of the language L here – one corresponding to each subset of $B_L = \{p, q\}$. The application of T_P to any of these interpretation yields the empty interpretation because no interpretation can satisfy $p \& \neg p$.

Lemma 3 There is only one pure logic program that is body inconsistent, viz. the empty program. $\square$

Theorem 3 (Ring Non-Extensibility Theorem) The bounded distributive lattice $\langle \Phi(L), \oplus, \otimes \rangle$ has no ring-extension iff B_L is non-empty. (This is the same as saying that $\langle \Phi(L), \oplus, \otimes \rangle$ has a ring extension iff $B_L = \emptyset$).

Proof. First, recall that $\Phi(L)$ is the set of T_P operators associated with the pure logic programs expressible in L .

($\leftarrow$) Suppose B_L is non-empty. Then there are at least two distinct T_P operators in $\Phi(L)$. (To see this, let $A \in B_L$. Such an A exists by our assumption that B_L is non-empty. Then set $P_0 = \{A \Leftarrow\}$ and $Q_0 = \{A \Leftarrow A\}$. Clearly $T_{P_0} \neq T_{Q_0}$.) We now show that the only pure logic program P such that T_P has a join inverse is $P = \emptyset$. Clearly, the empty program has a join inverse, viz. itself. Suppose now that P is some program having a join inverse Q . Then $(\forall I)(T_P \oplus T_Q)(I) = \emptyset$ and so $(\forall I)(T_P(I) \cup T_Q(I)) = \emptyset$, whence $(\forall I)[T_P(I) = \emptyset \text{ and } T_Q(I) = \emptyset]$. Hence, by Lemma 3 and as $B_L \neq \emptyset$, $P = \emptyset$. Thus, the only element in $\Phi(L)$ having a join inverse is the empty program. As there are at least two non-empty pure logic programs in L , there are pure logic programs in L having no join inverse. Hence, $\langle \Phi(L), \oplus, \otimes \rangle$ has no ring extension.

($\rightarrow$) Suppose $\langle \Phi(L), \oplus, \otimes \rangle$ has no ring extension and $B_L = \emptyset$. Then $\Phi(L)$ has only one element, viz. $\lambda I. \emptyset$. It is easy to verify that in this case, $\langle \Phi(L), \oplus, \otimes \rangle$ is a ring. $\square$

Theorem 4 (Second Ring Non-Extensibility Theorem.) The bounded distributive lattice $\langle \Psi(L), \oplus, \otimes \rangle$ has no ring extension iff B_L is non-empty.

Proof. ($\leftarrow$) Suppose B_L is non-empty. Then there are at least two programs P, Q in L such that $T_P \neq T_Q$ (take P, Q to be the programs constructed in the proof of Theorem 3). We now show that a general logic program R in L has a join inverse iff R is body inconsistent. Clearly,

R has a join inverse

iff there is a general program R' in L such that $(T_R \oplus T_{R'})(I) = \emptyset$ for all $I \subseteq B_L$

iff $T_R(I) \cup T_{R'}(I) = \emptyset$ for all $I \subseteq B_L$

iff $T_R(I) = \emptyset$ and $T_{R'}(I) = \emptyset$ for all $I \subseteq B_L$

iff R and R' are both body-inconsistent.

But both P and Q are programs in L that are body consistent. Hence, they have no join inverse. Thus, $\langle \Psi(L), \oplus, \otimes \rangle$ has no ring extension. This completes one part of the proof.

($\rightarrow$) Immediate. $\square$

The proofs of Theorem 3 and Theorem 4 tell us that the structure $\langle \Psi(L), \oplus \rangle$ is not a group in general. Indeed, it can similarly be shown that $\langle \Psi(L), \otimes \rangle$ in general, is not a group either.

Definition 4 An element $d \in D$ (D is a distributive lattice) is said to be *meet reducible* if there exist $x, y \in D$ such that $x \cdot y = d$ and x and y are different from d . If a distributive lattice has a

least element 0 and the 0 is meet reducible then such x and y are called *zero divisors* of D . We denote the set of zero divisors of this distributive lattice by $\mathcal{Z}(D)$.

Theorem 5 (First Zero Divisor Theorem.) The distributive lattice $\langle \Phi(L), \oplus, \otimes \rangle$ has zero divisors iff $\text{card}(B_L) \geq 2$.

Proof. ($\leftarrow$) Suppose $\text{card}(B_L) \geq 2$. Then there are distinct ground atoms $A, B \in B_L$. Let P be the program consisting of the single clause $A \leftarrow B$, and let Q be the program consisting of the single clause $B \leftarrow A$. Then it is easy to check (cf. the table below) that $T_P \neq 0$ and $T_Q \neq 0$, but $(T_P \otimes T_Q) = 0$.

I	$T_P(I)$	$T_Q(I)$	$(T_P \otimes T_Q)(I)$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$\{A\}$	$\emptyset$	$\{B\}$	$\emptyset$
$\{B\}$	$\{A\}$	$\emptyset$	$\emptyset$
$\{A, B\}$	$\{A\}$	$\{B\}$	$\emptyset$

($\rightarrow$) Assume that $\langle \Phi(L), \oplus, \otimes \rangle$ has zero divisors. Then there are at least two zero divisors (immediate from definition of zero divisors and as $\otimes$ is idempotent). Thus, there are at least two programs P, Q in L such that

- (I) $T_P \neq T_Q$ and
- (II) $T_P \otimes T_Q = 0$ and
- (III) $T_P(I) \neq \emptyset$ for some $I \subseteq B_L$ and
- (IV) $T_Q(J) \neq \emptyset$ for some $J \subseteq B_L$.

$\text{card}(B_L) \neq 0$ because then $\Phi(L) = \{0\}$ and there are no zero divisors which is a contradiction. We only need to show that $\text{card}(B_L) \neq 1$. We show how to derive a contradiction when we assume that $\text{card}(B_L) = 1$. If $\text{card}(B_L) = 1$, i.e. $B_L = \{A\}$ for some atom A , then by (III) and (IV) above, we must have that $T_P(I) = \{A\} = T_Q(J)$. Now, if $B_L = \{A\}$, then the only negation-free ground clauses expressible in L are:

- $C1 : \quad A \leftarrow$
- $C2 : \quad A \leftarrow A$

Thus³, $\text{grad}(P)$ and $\text{grad}(Q)$ must be one of the following three programs: $\{C1\}$, $\{C2\}$, and $\{C1, C2\}$. (The empty program is excluded by conditions (III) and (IV) above). In all cases, we can derive a contradiction as follows.

Case 1. ($I = J$) In that case, $(T_P \otimes T_Q)(I) = T_P(I) \cap T_Q(I) = T_P(I) \cap T_Q(J) = \{A\} \neq \emptyset$. Hence, $(T_P \otimes T_Q) \neq 0$. A contradiction.

Case 2. ($I \neq J$). There are two subcases.

³We assume that no atom occurs twice in a ground clause, i.e. the clause $A \leftarrow A \& A$ is not considered as it exhibits the same properties (in terms of the T_P operator) as $A \leftarrow A$.

Subcase 2.1 ($I = \emptyset$ and $J = \{A\}$) As $T_P(I) = \{A\}$, we know that $C1$ is in $\text{grad}(P)$. Then $(T_P \otimes T_Q)(J) = \{A\} \neq \emptyset$. A contradiction.

Subcase 2.2 ($I = \{A\}$ and $J = \emptyset$) This subcase is symmetric to Subcase 2.1.

Hence it must be the case that $\text{card}(B_L) \geq 2$. $\square$

Note that the above theorem does not hold for $\langle \Psi(L), \oplus, \otimes \rangle$. To see this consider the following example.

Example 3 Let P be the program consisting of the single clause:

$$A \Leftarrow A$$

while Q consists of the single clause:

$$A \Leftarrow \neg A$$

where B_L consists of the single atom A . Then $T_P \neq \mathbf{0}$ because $T_P(\{A\}) = \{A\} \neq \emptyset$; similarly, $T_Q \neq \mathbf{0}$ because $T_Q(\emptyset) = \{A\} \neq \emptyset$. But $(T_P \otimes T_Q) = \mathbf{0}$ (see table below).

I	$T_P(I)$	$T_Q(I)$	$(T_P \otimes T_Q)(I)$
$\emptyset$	$\emptyset$	$\{A\}$	$\emptyset$
$\{A\}$	$\{A\}$	$\emptyset$	$\emptyset$

Thus, when we consider general logic programs, we may have zero divisors of $\langle \Psi(L), \oplus, \otimes \rangle$ even when B_L contains only one element.

Thus, we are led to the question: *Exactly what kind of programs have associated operators that are zero divisors of the bounded distributive lattice $\langle \Psi(L), \oplus, \otimes \rangle$?*

Definition 5 Two general logic programs P, Q are said to be *contravening* iff whenever C_1 is a clause in P and C_2 is a clause in Q of the form:

$$A_1 \Leftarrow \text{Body1}$$

$$A_2 \Leftarrow \text{Body2}$$

and A_1 and A_2 are unifiable via mgu Θ , then

$$(\exists u_1, \dots, u_s)(\exists v_1, \dots, v_t)(\exists x_1, \dots, x_k)(\exists y_1, \dots, y_n)(\text{Body1} \& \text{Body2})\Theta$$

has no Herbrand model. Here, $u_1, \dots, u_s$ are all variables that occur in $A_1\Theta$; $x_1, \dots, x_k$ are all other variables that occur in $\text{Body1}\Theta$. The v_i 's and y_j 's play a similar role for C_2 .

Example 4 For instance, let L contain one constant symbol, a , and two unary predicate symbols p, q and no other non-variable symbols. Let P be the single clause program:

$$p(X) \Leftarrow q(Y)$$

and let Q consist of the single clause:

$$p(a) \Leftarrow \neg q(a)$$

Then X/a is the mgu of the head of these two clauses, and $(\exists Y)(q(Y) \& \neg q(a))$ has no Herbrand model (w.r.t. L). [Remember that B_L has only two atoms: $p(a)$ and $q(a)$]. Thus, P and Q are contravening w.r.t. L . Now, if L' consists of L together with the additional constant b , then P and Q are not contravening. Thus, contravention depends crucially on the language we are considering.

Example 5 Let $B_L = \{p(a), p(b)\}$. Let P, Q be the programs:

$$P : \quad \{p(a) \Leftarrow p(b)\}$$

$$Q : \quad \{p(b) \Leftarrow p(a)\}$$

(Notice that in effect, these programs are the same ones used in the first part of the proof of Theorem 5.) Then P and Q are trivially contravening as no two clause heads unify.

Mancarella and Pedreschi [17] define an operation $@$ on programs as follows: If

$$A_1 \Leftarrow Body1$$

and

$$A_2 \Leftarrow Body2$$

are clauses in P, Q respectively (standardized apart), and A_1, A_2 are unifiable via mgu θ , then the clause

$$A_1\theta \Leftarrow (Body1 \& Body2)\theta$$

is in $P@Q$. Nothing else is in $P@Q$. The following result is easy to establish.

Proposition 2 Suppose P, Q are general logic programs. Then:

$$1. T_P \otimes T_Q = T_{P@Q}.$$

$$2. T_P \oplus T_Q = T_{P \cup Q}.$$

□

Mancarella and Pedreschi [17] proved the above proposition for pure logic programs. This restriction is not necessary.

Theorem 6 (Second Zero Divisor Theorem.) T_P is a zero divisor of $\langle \Psi(L), \oplus, \otimes \rangle$ iff P is not body-inconsistent and there is a general logic program Q which is not body-inconsistent such that $grd(P)$ and $grd(Q)$ are contravening.

Proof. T_P is a zero divisor of $\langle \Psi(L), \oplus, \otimes \rangle$

iff $T_P \neq 0$ and there is a $T_Q \neq 0$ such that $(T_P \otimes T_Q) = 0$

iff P is not body inconsistent and there is a Q that is not body inconsistent such that $(T_P \otimes T_Q) = 0$
(by Lemma 2)

iff P is not body inconsistent and there is a non-body-inconsistent Q such that $(\forall I)(T_P(I) \cap T_Q(I)) = \emptyset$ (by definition of $\otimes$)

iff P is body consistent and there is a body consistent Q is body consistent and for all $I \subseteq B_L$, $T_{P \otimes Q}(I) = \emptyset$

iff P is body consistent and there is a body consistent Q and $P \otimes Q$ is body inconsistent

iff P is body consistent and there is a Q that is body consistent and for all $I \subseteq B_L$, whenever $A \leftarrow \text{Body1} \& \text{Body2}$ is in $\text{grad}(P \otimes Q)$ (where $A \leftarrow \text{Body1}$ is in $\text{grad}(P)$ and where $A \leftarrow \text{Body2}$ is in $\text{grad}(Q)$) it is the case that $I \not\models (\text{Body1} \& \text{Body2})$.

iff $\text{grad}(P)$ is not body inconsistent and there is a Q such that $\text{grad}(Q)$ is not body inconsistent and such that $\text{grad}(P)$ and $\text{grad}(Q)$ are contravening. $\square$

When we set up an algebraic system, we must understand exactly how much “power” our algebraic system gives us. Given a program P , what kinds of programs can we get by repeated applications of the algebraic operations? Can these repeated applications completely annihilate P ? (i.e. given that $T_P \neq 0$, is it possible to repeatedly apply the algebraic operations to P and some other program Q to derive a program P' such that $T_{P'} = 0$?) It is precisely to understand such issues that we have studied zero divisors.

Lemma 4 If P is a general logic program, then there are general logic programs P_1, P_2 such that $P = P_1 \otimes P_2$. $\square$

Lemma 5 $\langle \Psi(L), \oplus, \otimes \rangle$ always has at least one maximal ideal. (Recall that the set $\Psi(L)$ is excluded from being a maximal ideal of $\langle \Psi(L), \oplus, \otimes \rangle$ by definition).

Proof. Immediate consequence of Zorn’s Lemma. $\square$

Lemma 6 Every proper ideal J of $\langle \Psi(L), \oplus, \otimes \rangle$ is contained in a maximal ideal. $\square$

Lemma 7 The only unit (recall that a unit is an element with a meet inverse) of $\langle \Psi(L), \oplus, \otimes \rangle$ is 1.

Proof. $T_P \in \Psi(L)$ is a unit

iff there is a logic program Q in L such that $(T_P \otimes T_Q) = 1$

iff $(\forall I) T_P(I) \cap T_Q(I) = B_L$

iff $T_P = T_Q = 1$. $\square$

Lemma 8 Let P be a general logic program in L . P is body-inconsistent iff $T_{P \otimes Q} = 0$ for all programs Q . $\square$

Definition 6 Suppose C_1 and C_2 are the general clauses below:

$$C_1: \quad A \Leftarrow L_1 \& \cdots \& L_n$$

$$C_2: \quad B \Leftarrow L'_1 \& \cdots \& L'_m$$

Then C_1 *subsumes* C_2 (via θ) iff there is a most general substitution θ such that:

1. $A\theta = B$ and
2. $\{L_1\theta, \dots, L_n\theta\} \subseteq \{L'_1, \dots, L'_m\}$.

Definition 7 Suppose P, Q are general logic programs. P *subsumes* Q iff each clause in Q is subsumed by some clause in P . P and Q are *subsumption equivalent* iff P subsumes Q and Q subsumes P .

If P is a logic program and L some language then we use the notation $grd(P)$ to denote the (possibly infinite) program consisting of all ground instances (w.r.t. L) of all clauses in P .

Lemma 9 Suppose P, Q are general logic programs in language L . If P and Q are subsumption equivalent then $grd(P)$ and $grd(Q)$ are subsumption equivalent. $\square$

Lemma 10 Suppose C_1 and C_2 are clauses. If C_1 subsumes C_2 , then $T_{C_1} \geq T_{C_2}$.

Proof. Suppose $A \in T_{C_2}(I)$. Then

$$A \Leftarrow B_1 \& \cdots \& B_n \& \neg D_1 \& \cdots \& \neg D_m$$

is a ground instance $C_2\sigma$ of C_2 such that

$$I \models B_1 \& \cdots \& B_n \& \neg D_1 \& \cdots \& \neg D_m.$$

As C_1 subsumes C_2 , it now follows immediately from the definition of subsumption, that $A \in T_{C_1}(I)$.

$\square$

Corollary 1 Suppose P and Q are general logic programs. If Q subsumes P , then $T_Q \geq T_P$. $\square$

Definition 8 Suppose C_1, C_2 are the following clauses respectively:

$$C_1: \quad A \Leftarrow B_1 \& \cdots \& B_n \& \neg D_1 \& \cdots \& \neg D_m$$

$$C_2: \quad A' \Leftarrow B'_1 \& \cdots \& B'_r \& \neg D'_1 \& \cdots \& \neg D'_s$$

We say C_1 *weakly subsumes* C_2 iff there is a substitution θ such that:

1. $A' = A\theta$ and
2. $\{B_1\theta, \dots, B_n\theta\} \subseteq \{B'_1, \dots, B'_r\}$.

Program P weakly subsumes program Q iff each clause in Q is weakly subsumed by some clause in P . Programs P and Q are weakly subsumption equivalent iff P weakly subsumes Q and Q weakly subsumes P .

Theorem 7 Suppose $grad(P)$ and $grad(Q)$ are body consistent. Then:

1. if $grad(P)$ and $grad(Q)$ are subsumption equivalent, then $T_P = T_Q$.
2. if $T_P = T_Q$ then $grad(P)$ and $grad(Q)$ are weakly subsumption equivalent.

Proof. 1) Immediate consequence of Corollary 1 and the fact that $T_P = T_{grad(P)}$ and $T_Q = T_{grad(Q)}$.
 2) Suppose $T_P = T_Q$. We show below that if C_p is in $grad(P)$, then there is some C_q in $grad(Q)$ such that C_q weakly subsumes C_p . Suppose $C_p \equiv$

$$A \leftarrow B_1 \& \dots \& B_n \& \neg D_1 \& \dots \& \neg D_m.$$

Then, as $grad(P)$ is body consistent, there is an interpretation $I \subseteq B_L$ such that

$$I \models B_1 \& \dots \& B_n \& \neg D_1 \& \dots \& \neg D_m.$$

We may assume, without loss of generality, that I is minimal w.r.t. inclusion. Then, as $T_{grad(P)} = T_{grad(Q)}$, there is a clause $C_q \equiv$

$$A \leftarrow B'_1 \& \dots \& B'_r \& \neg D'_1 \& \dots \& \neg D'_s$$

in $grad(Q)$ such that

$$I \models B'_1 \& \dots \& B'_r \& \neg D'_1 \& \dots \& \neg D'_s$$

As I is minimal w.r.t. inclusion,

$$\{B'_1, \dots, B'_r\} \subseteq \{B_1, \dots, B_n\}.$$

Thus, C_q weakly subsumes C_p . Thus we have shown that Q weakly subsumes P . Showing that P weakly subsumes Q is symmetric. $\square$

Maher[14] proved a similar result for pure logic programs. Theorem 7 above extends Maher's result to the case of general logic programs.

Subsumption equivalence of P and Q implies that $T_P = T_Q$. There is no need for the assumption that P and Q are body consistent.

However to show that $T_P = T_Q$ implies that P and Q are weakly subsumption equivalent, we need the assumption that P and Q are body consistent. To see this, let $P = \{r \leftarrow s \& \neg s\}$ and $Q = \emptyset$. Then $T_P = \mathbf{0} = T_Q$, but clearly P and Q are not subsumption equivalent. On the other hand, the suspicious reader may wonder what happens if we change P to $P' = \{r \leftarrow s \& \neg w, w \leftarrow s, s \leftarrow w\}$. This says that s and w are essentially the same thing. But here, $T_{P'} \neq \mathbf{0}$ because $T_{P'}(\{s\}) = \{w, r\}$ – hence, $T_{P'}$ is certainly not $\mathbf{0}$. So this time, it is not the case that $T_{P'} = \mathbf{0}$ and so $T_{P'} \neq T_Q$. And clearly, P' and Q are not subsumption-equivalent.

Having proved many technical theorems concerning the distributive lattices $\langle \Psi(L), \oplus, \otimes \rangle$ and $\langle \Phi(L), \oplus, \otimes \rangle$, let us step back and take a look at what *the algebra of logic programs* really is (recall that this is the “catch” phrase in the titles of at least two published papers, viz. [17] and [18]) all about.

We are interested in seeing how to put two given programs together. This may be done in several different ways. Selection of the *admissible* ways of composing programs together yields an algebra of logic programs. Mancarella and Pedreschi selected two ways, viz. $\oplus$ and $\otimes$. They were the first to give a coherent treatment of the algebraic foundations of the semantics of composing logic programs. As in any first attempt, they concentrated on laying the basic foundation. Our attempt is to dig deeper into these foundations and study the properties of the Mancarella-Pedreschi Algebra [17].

Given a finite collection $\mathcal{C}$ of logic program operators (i.e. $\mathcal{C} \subseteq \Psi(L)$), what is the ideal I they generate? Precisely the set of all operators associated with programs that are obtained by applying the admissible operators of our algebra finitely many times to the generators in $\mathcal{C}$. Thus, given an *initial* set of programs, the ideals generated by this set correspond to the set of all programs that can be obtained from the initial set of programs by applying the admissible operations for composing programs finitely many times.

3 Conclusions

Of late, there has been considerable interest in studying compositional properties of logic programs. Fitting [8], for instance has taken a recursion-theoretic approach to the problem. Miller [15, 16] has taken an intuitionistic approach while Mancarella and Pedreschi have taken an algebraic approach [17].

We believe that the algebraic approach has many virtues. Chief among them is *simplicity* and *naturalness*. What does it mean to *compose* two logic programs? The natural answer to this is that the programs are conjoined via an operator. What is this operator? Again, the natural answer is that this operator is a map from programs to programs. Algebra is the study of a structure $\langle A, op1, op2, \dots, opn \rangle$ where $op1, \dots, opn$ are operators on A (of possibly different arities). The study of logic program compositionality is just the study of the space of logic programs and the operators that conjoin these programs. Thus, we believe that the algebraic framework provides the right setting to study compositionality.

In this paper, we have significantly improved upon the initial results of Mancarella and Pedreschi [17] in the following directions:

1. we have extended their approach to the case of general logic programs (they considered pure logic programs only)
2. We have proved important non-extendibility theorems and shown that the algebraic structure $\langle \Psi(L), \oplus, \otimes \rangle$ is a bounded distributive lattice
3. we prove important properties about the ideals and zero divisors of this algebra of programs.

4. we link this approach to existing research on logic program equivalence (cf. Maher [14]), cf. Theorem 7.

Moreover, all the above apply to general logic programs rather than only to pure logic programs as in the case of Mancarella and Pedreschi [17].

This is the first of an n -part ($n \geq 3$) series of papers on the algebraic foundations of logic programming. In part II of this series, we expect to generalize the results contained here to multi-valued and paraconsistent logic programs. In part III, we expect to study the algebraic properties of the space of reachable models of classical logic programs. This is done by means of the Stone Representation theorem which allows us to use the full force of our topological characterization (cf. [3,4]) of the space of reachable models of logic programs and their completions.

References

- [1] M. F. Atiyah and I. G. Macdonald. (1969) *Introduction to Commutative Algebra*, Addison Wesley.
- [2] A. Batarekh and V.S. Subrahmanian. (1988) *Semantical Equivalences of (Non-Classical) Logic Programs*, in: Proc. 5th International Conference/Symposium on Logic Programming, (eds. R. Kowalski and K. Bowen), pps 960-977, MIT Press.
- [3] A. Batarekh and V. S. Subrahmanian. (1988) *Topological Model Set Deformations in Logic Programming*, to appear in: *Fundamenta Informaticae*.
- [4] A. Batarekh and V. S. Subrahmanian. (1988) *The Query Topology in Logic Programming*, in: Proc. of the International Symposium on Theoretical Aspects of Computer Science, Lecture Notes in Computer Science Vol. 349, pps 375-387, Springer Verlag.
- [5] H. A. Blair. (1986) *Decidability in the Herbrand Base*, Proc. of the Workshop on Foundations of Deductive Databases and Logic Programming, (ed. Jack Minker). Washington DC, Aug. 1986.
- [6] H. A. Blair and V. S. Subrahmanian. (1987) *Paraconsistent Logic Programming*, in: Lecture Notes in Computer Science, Vol. 287, pps 340-360. Extended version to appear in: *Theoretical Computer Science*.
- [7] H. A. Blair and V. S. Subrahmanian. (1988) *Paraconsistent Foundations for Logic Programming*, to appear in: *Journal of Non-Classical Logic*.
- [8] M. C. Fitting. (1987) *Enumeration Operators and Modular Logic Programming*, *Journal of Logic Programming*, 4, pps 11-21.
- [9] M. C. Fitting. (1988) *Logic Programming on a Topological Bilattice*, *Fundamenta Informaticae*, 11, pps209-218.
- [10] M. C. Fitting. (1988) *Bilattices and the Semantics of Logic Programming*, manuscript.

- [11] G. Grätzer. (1971) *Lattice Theory: First Concepts and Distributive Lattices*, W. H. Freeman and Company.
- [12] L. Henkin, J. D. Monk and A. Tarski. (1971) *Cylindric Algebras, Part I*, North Holland.
- [13] J. W. Lloyd. (1984) *Foundations of Logic Programming*, Springer Verlag.
- [14] M. J. Maher. (1988) *Equivalences of Logic Programs*, in: Foundations of Deductive Databases and Logic Programming, ed. Jack Minker, Morgan-Kaufman.
- [15] D. Miller. (1986) *A Theory of Modules for Logic Programming*, Proc. 3rd IEEE Symposium in Logic Programming, (Salt Lake City, Utah), pps 106–115.
- [16] D. Miller. (1987) *A Logical Analysis of Modules in Logic*, to appear in: Journal of Logic Programming.
- [17] P. Mancarella and D. Pedreschi. (1988) *An Algebra of Logic Programs*, in: Proc. 5th International Conference on Logic Programming (eds. R. Kowalski and K. Bowen), pps 1006-1023, MIT Press.
- [18] R. O’Keefe. (1985) *Towards an Algebra for Constructing Logic Programs*, Proc. 2nd IEEE Symposium on Logic Programming (Boston, MA), pps 152–160.
- [19] R. Sikorski. (1964) *Boolean Algebras*, Springer Verlag.
- [20] V. S. Subrahmanian. (1988) *Mechanical Proof Procedures for Many-Valued Lattice Based Logic Programming*, to appear in: Journal of Non-Classical Logic.